

TD7 - L'héritage en programmation objet

1) Problématique	1
2) Questions	1
3) Conclusion	14

1) Problématique

Nous cherchons à comprendre le fonctionnement du mécanisme d'héritage de la POO, pour cela nous allons répondre à plusieurs questions en rapport avec ce mécanisme.

2) Questions

Question 1 : Donner un exemple d'héritage.

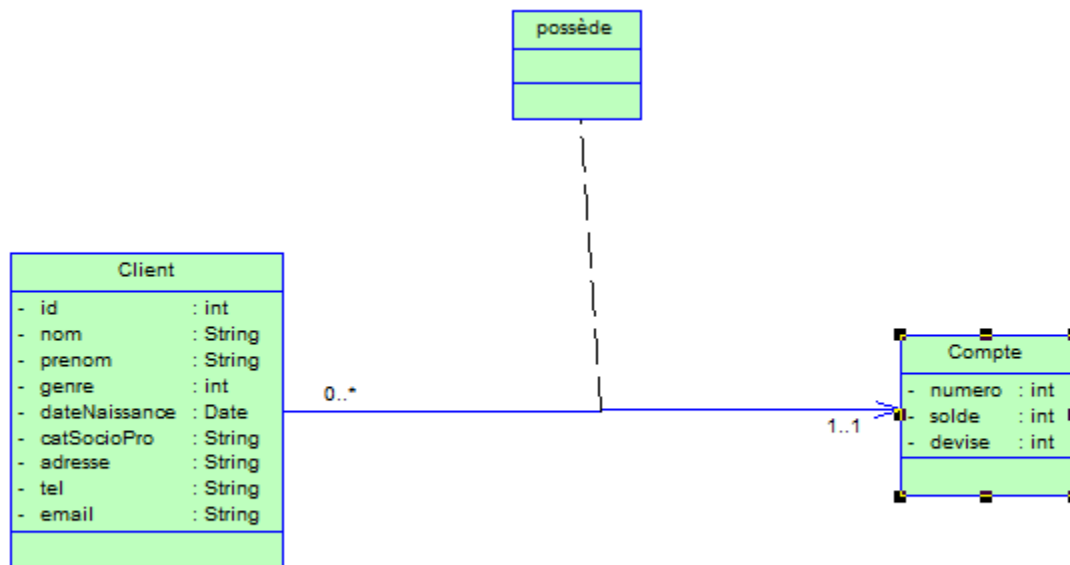
Imaginons une classe Animal, nous pouvons créer une classe enfant qui héritera de Animal et qui sera donc capable de réutiliser ou de remplacer les méthodes de la classe Animal. Voici l'exemple sous forme de code Java :

```
// Classe parent
class Animal {
    void parler() {
        System.out.println("Je fais un bruit");
    }
}

// Classe enfant
class Chien extends Animal {
    void parler() {
        System.out.println("Woof !");
    }
}

public class Main {
    public static void main(String[] args) {
        Chien monChien = new Chien();
        monChien.parler(); // Affiche "Woof !"
    }
}
```

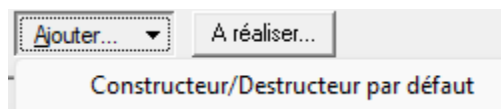
Question 2 : Faire le diagramme de classe de la banque.



Question 3 : Faire le constructeur, accesseurs et mutateur de la classe compte.

Pour faire cela c'est simple, il suffit de sélectionner l'attribut que l'on souhaite utiliser pour faire un setter et un getter approprié à celui-ci et cliquer sur .

Pour le constructeur, il suffit de cliquer sur **Ajouter** dans la section **Opérations** et de cliquer sur **Constructeur/Destructeur par défaut** :



Et lorsque cela est fait il suffit d'ajouter les paramètres au constructeur en double cliquant sur celui-ci, en allant dans paramètres et en remplissant les champs de façon adapté comme ici :

1	p_numero	pNumero	int	Entrée
2	p_solde	pSolde	int	Entrée
3	p_devis	pDevis	String	Entrée

Question 4 : Ecrire la méthode débiter et créditer

Pour créer les méthodes nous allons aller dans **Opérations** et ajouter aux deux nouvelles lignes les méthodes debiter et crediter :

8	debiter	debiter	void	public
9	crediter	crediter	void	public

Il ne faut pas oublier d'ajouter pour chacun de ces deux attributs le paramètre **montant** :

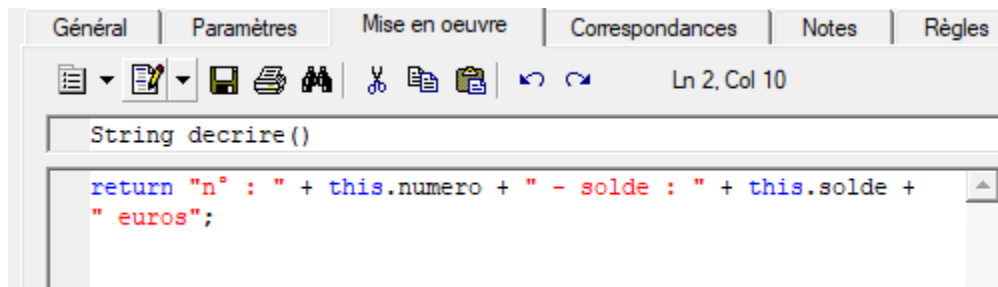
1	montant	montant	int	Entrée
---	---------	---------	-----	--------

Question 5 : Ecrire la méthode decrire de la classe compte qui retourne une chaîne de caractere avec la description d'un compte.

Pour faire cela on fait comme avant on ajoute la ligne decrire :

→	decrire	decrire	String	public
---	---------	---------	--------	--------

Puis nous allons aller dans l'onglet Mise en oeuvre après avoir double cliqué sur la ligne decrire et ajouter le code de la méthode :

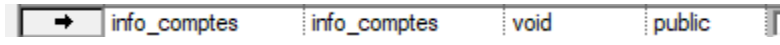


```
String decrire()
{
    return "n° : " + this.numero + " - solde : " + this.solde + " euros";
}
```

(Il ne faut pas oublier d'appliquer pour chaque modification)

Question 6 : Création de info_comptes

Pour écrire la méthode **info_comptes** qui permettra d'afficher l'ensemble des comptes d'un adhérent, nous allons aller dans la table client, dans la section opération et nous allons ajouter la méthode de cette façon :

 , puis on applique.

Ensuite comme dans la question d'auparavant nous allons cliquer sur la section **Mise en oeuvre** présente dans les propriétés de la méthode et nous allons ajouter le code suivant :

```
System.out.println("Liste des comptes de M. " + this.nom + " " + this.prenom);  
  
// Nous avons au préalable nommé le rôle de la classe Compte en sesComptes  
for (Compte unCompte : this.sesComptes) {  
    System.out.println(unCompte.Decrire());  
}
```

Nous utilisons la méthode decrire pour l'ensemble des comptes et nous affichons la liste des comptes présentes dans l'arraylist caché "sesComptes" que l'on a nommé en changeant le nom de rôle de la classe Compte dans ses propriétés.

Question 7 : Affichage de la liste des comptes de Martin

Pour la question 7 j'ai dû générer les codes java avec PowerAMC puis j'ai mis le code de la classe compte dans compte sur bluej et celui de classe client dans client, puis j'ai créé un main où l'on utilise le constructeur de la classe client (que j'ai dû générer au passage dans le diagramme de classe). Le résultat est le suivant :

```
Affichage de la liste des comptes de Martin :  
n° : 13245411324 - solde : 2548200.0 €  
n° : 32544771289 - solde : 1000.0 €  
n° : 22544771289 - solde : -214.0 €
```

Question 8 : Tableau des attributs identiques et différent entre le compte épargne et le compte “classique”

Liste des attributs identiques entre le compte épargne et un compte “classique”	Liste des attributs différent entre le compte épargne et un compte “classique”
Numero Titulaire Solde Devise	Taux_interet

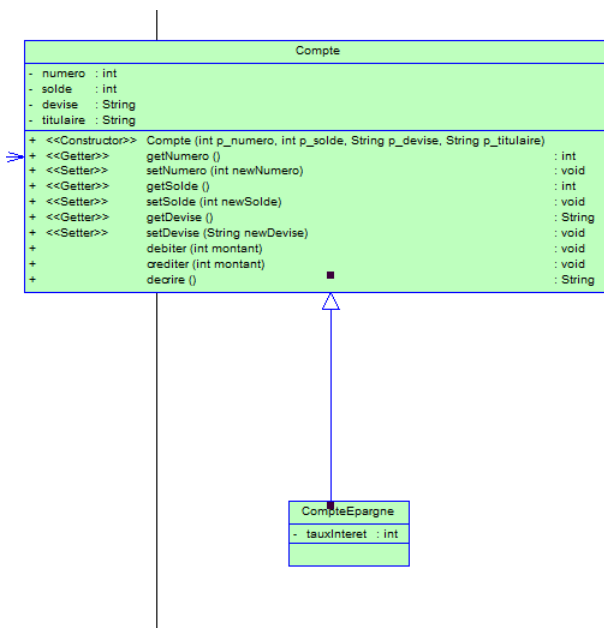
Question 9 : Création de classe fille “CompteEpargne”

Pour la question 9 nous allons ajouter la classe fille de Compte qui s'appellera CompteEpargne. Compte sera donc la classe mère.

Pour faire le système d'héritage nous allons utiliser l'outil **Généralisation**



et nous allons cliquer sur CompteEpargne et glisser jusqu'à Compte pour faire le lien parent-enfant.



Question 10 : Création du constructeur + getters/setters

Nous allons ajouter titulaire, solde et numero aux attributs de la nouvelle classe puis nous allons générer les getters/setters de chaque attributs et enfin nous allons ajouter dans les paramètres du constructeur chaque attribut pour avoir le résultat suivant :

CompteEpargne		
- tauxInteret	: int	
- numero	: int	
- solde	: int	
- titulaire	: String	
+ <<Getter>>	getTauxInteret ()	: int
+ <<Setter>>	setTauxInteret (int newTauxInteret)	: void
+ <<Constructor>>	CompteEpargne (int taux, int numero, int solde, String titulaire)	
+ <<Getter>>	getNumero ()	: int
+ <<Setter>>	setNumero (int newNumero)	: void
+ <<Getter>>	getSolde ()	: int
+ <<Setter>>	setSolde (int newSolde)	: void
+ <<Getter>>	getTitulaire ()	: String
+ <<Setter>>	setTitulaire (String newTitulaire)	: void

Question 11 : Création d'une nouvelle méthode decire

Pour cette question nous allons créer une nouvelle méthode decire dans laquelle nous allons rajouter le taux d'intérêt dans la section **Mise en oeuvre** avec ce code :

```
Propriétés de l'opération - decire (decire)
Général Paramètres Mise en oeuvre Correspondances Notes Règles Diagrammes associés Attributs étendus Dépendances Version
Ln 1, Col 115
void decire ()
return "Compte Epargne n° : " + getNumero () + " - solde : " + getSolde () + " euros - taux = " + tauxInteret + "%";
```

Question 12 : Création de la méthode ajouter_intéret

Nous allons créer la méthode ajouter_intéret dans laquelle nous allons déclarer l'attribut interets qui s'appliquera au solde si celui-ci est positif :

```
void ajouter_intéret()
{
    if (getSolde() > 0) {
        double interets = getSolde() * (tauxInteret / 100);
        setSolde(getSolde() + interets);
    }
}
```

Question 13 et 14 : Etats des comptes avant et après intérêts

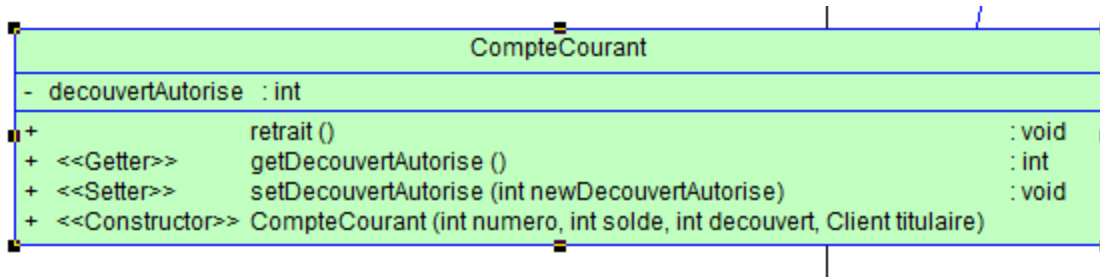
En ajoutant la classe de CompteEpargne dans BlueJ et en modifiant deux trois choses dans les autres classes nous pouvons faire en sorte d'afficher la liste des comptes du client avec le compte épargne en plus sur lequel on pourra voir le solde avant les intérêts et après les intérêts :

```
--- Etat des comptes avant intérêts ---
Client : Benjamin Martin
- Compte Classique n° : 13245411324 | Solde : 2548200.0 €
- Compte Epargne n° : 32544771289 | Solde : 1000.0 € (Taux : 5.0%)
- Compte Classique n° : 22544771289 | Solde : -214.0 €

--- Etat des comptes après intérêts ---
Client : Benjamin Martin
- Compte Classique n° : 13245411324 | Solde : 2548200.0 €
- Compte Epargne n° : 32544771289 | Solde : 1050.0 € (Taux : 5.0%)
- Compte Classique n° : 22544771289 | Solde : -214.0 €
```

Question 15 : Ajout de la classe compte courant

Pour la nouvelle classe compte courant, nous allons ajouter l'attribut **decouvertAutorise** et une méthode de retrait nommé **retrait** qui vérifiera si le solde est égal ou supérieur au découvert autorisé. Voici à quoi ressemble le diagramme :



Pour la méthode retrait voici le code utilisé :

```

void retrait()

public void retrait(int unMontant) {
    if ((this.solde - unMontant) >= -this.decouvertAutorise) {
        this.solde = this.solde - unMontant;
    } else {
        System.out.println("Erreur : Retrait impossible, dépassement du découvert au
    }
}
  
```

Malheureusement je n'ai pas réussi à faire marcher le système de compte courant sur BlueJ, donc je passe à la prochaine question.

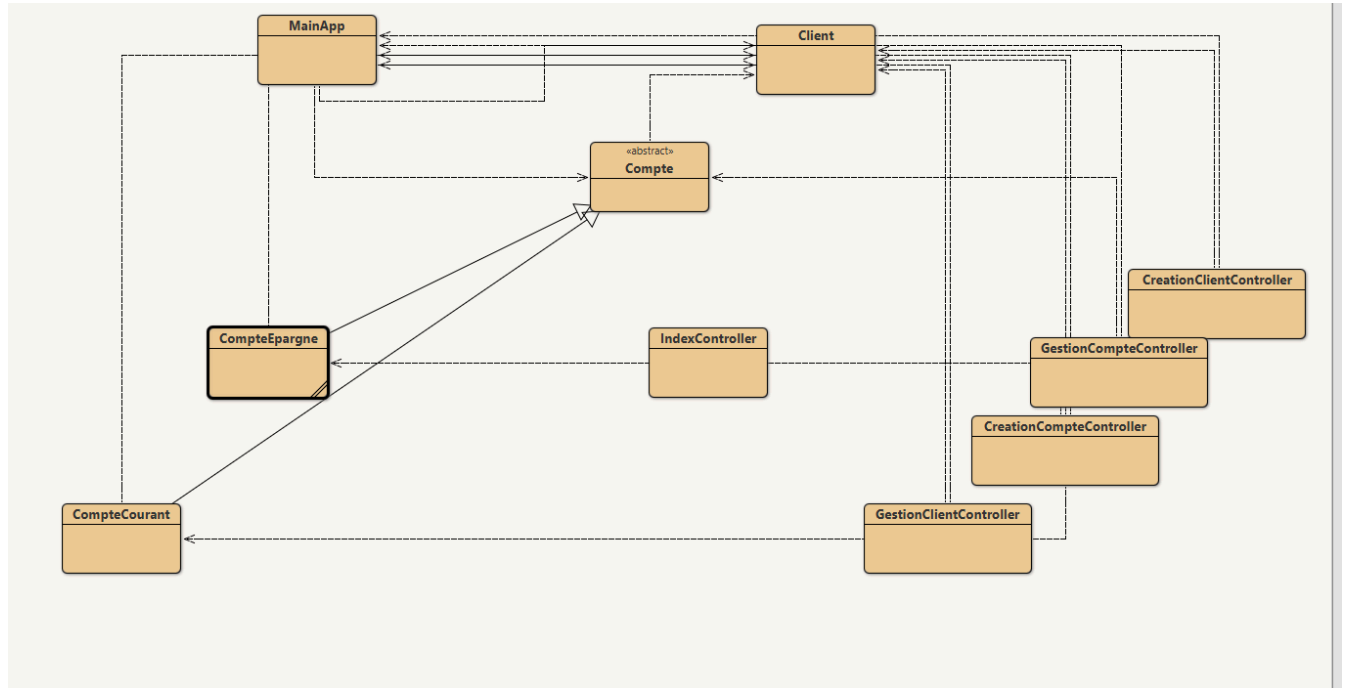
Question 16 : Scene Builder

Pour cette exercice nous allons utiliser SceneBuilder au lieu de WindowsBuilder, dans notre projet BlueJ nous allons créer 5 contrôleurs qui permettent chacun de faire marcher chaque page correctement, les 5 controlleurs sont : IndexController (page par défaut), CreationCompteController (page de création de compte), GestionCompteController (page de gestion du compte), CreationClientController (page de création de client) et GestionClientController (page de gestion du client).

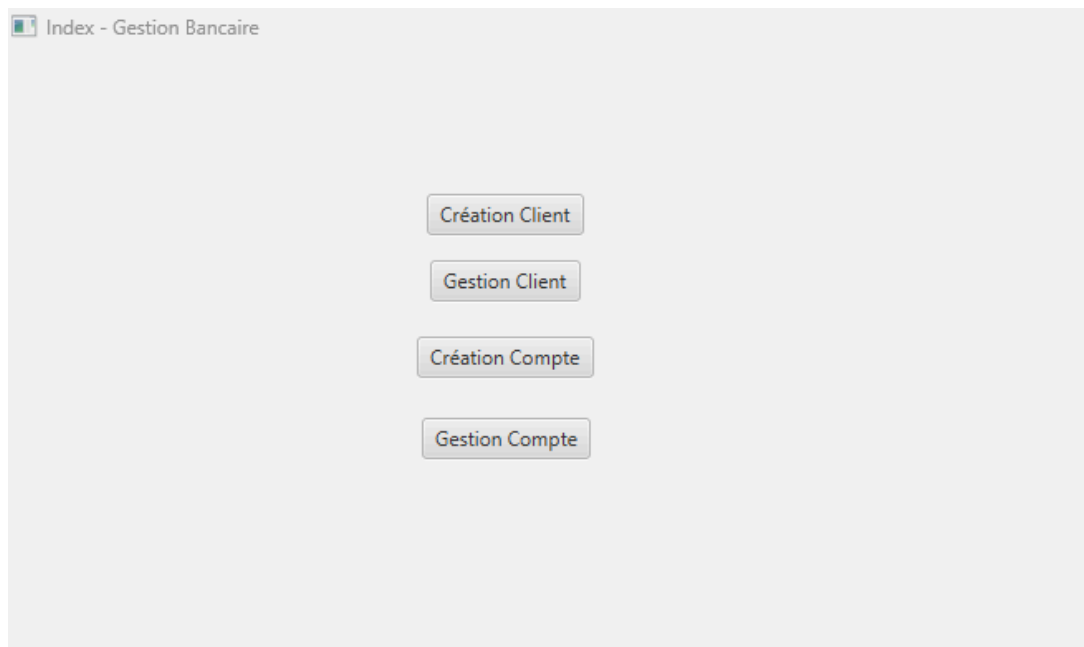
Il y a ensuite les classes pour les différents types de comptes : Compte courant (classe CompteCourant), compte épargne (classe CompteEpargne) et compte "classique" (classe Compte).

La dernière classe permet de pouvoir lancer l'application, elle s'appelle MainApp.

Voici une visualisation des classes :

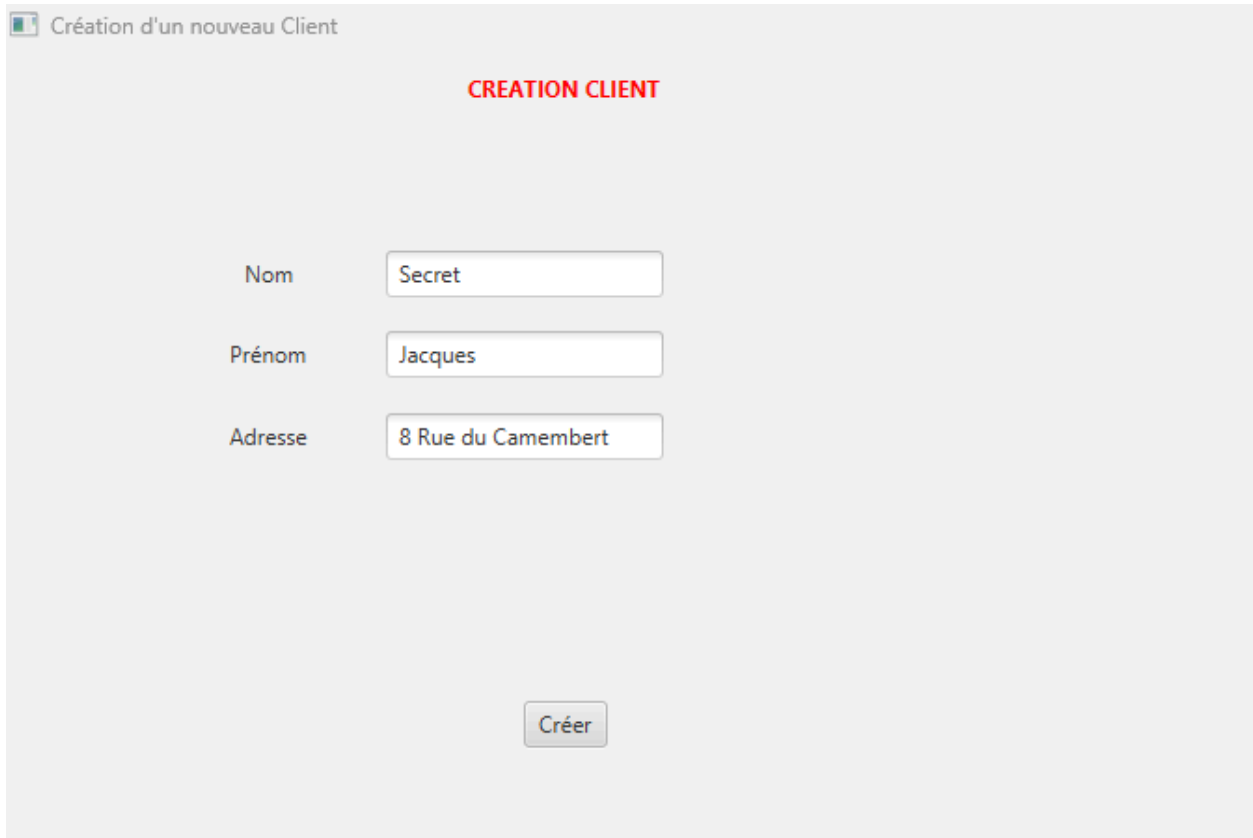


Donc voici le fonctionnement du logiciel, tout d'abord lorsqu'on lance le logiciel, la page d'index apparaît :



Sur la page d'index nous pouvons créer des clients, changer leur adresse si nécessaire, leur créer un compte et le gérer quand il le faut.

Nous allons tout d'abord créer un client nommé Jacques Secret qui vit 8 Rue du Camembert :



Création d'un nouveau Client

CREATION CLIENT

Nom

Prénom

Adresse

Lorsque tout est rempli on peut cliquer sur “Créer”. Notre nouveau client ira donc se placer dans une ArrayList (je n’ai pas fait de BDD pour cette exercice).

Lorsque le client est créé, nous pouvons gérer son adresse si nécessaire, nous allons supposer que le client a changé d’adresse et que maintenant il vit au 45 Rue de l’Emmental, nous pouvons donc retourner en arrière en fermant la page de création client et en cliquant sur le bouton “Gestion Client”, lorsque c’est fait nous pouvons changer l’adresse et cliquer sur “Modifier l’adresse” (il faut bien sélectionner le client pour que ça marche) :

Gestion des Clients

GESTION CLIENT

Secret Jacques - Adresse : 8 Rue du

Nouvelle adresse

45 Rue de l'Emmental

Modifier l'adresse

Comme on peut le voir, cela va changer l'adresse actuelle :

- Adresse : 45 Rue de l'Emmental

Maintenant, nous allons créer un compte pour Jacques Secret. Nous allons revenir sur la page d'index et cliquer sur "Création de compte", dans le menu déroulant nous allons sélectionner le client "Jacques Secret", nous allons lui donner un numéro de carte aléatoire, sélectionner le type de compte, indiquer le taux/découvert max selon le type de compte, ajouter un solde puis cliquer sur Créer. Dans notre cas Jacques Secret aura un compte courant :

Création de Compte

CREATION DE COMPTE

Sélectionner client : Secret Jacques ▼

Type de compte ☒ Compte courant ☐ Compte epargne

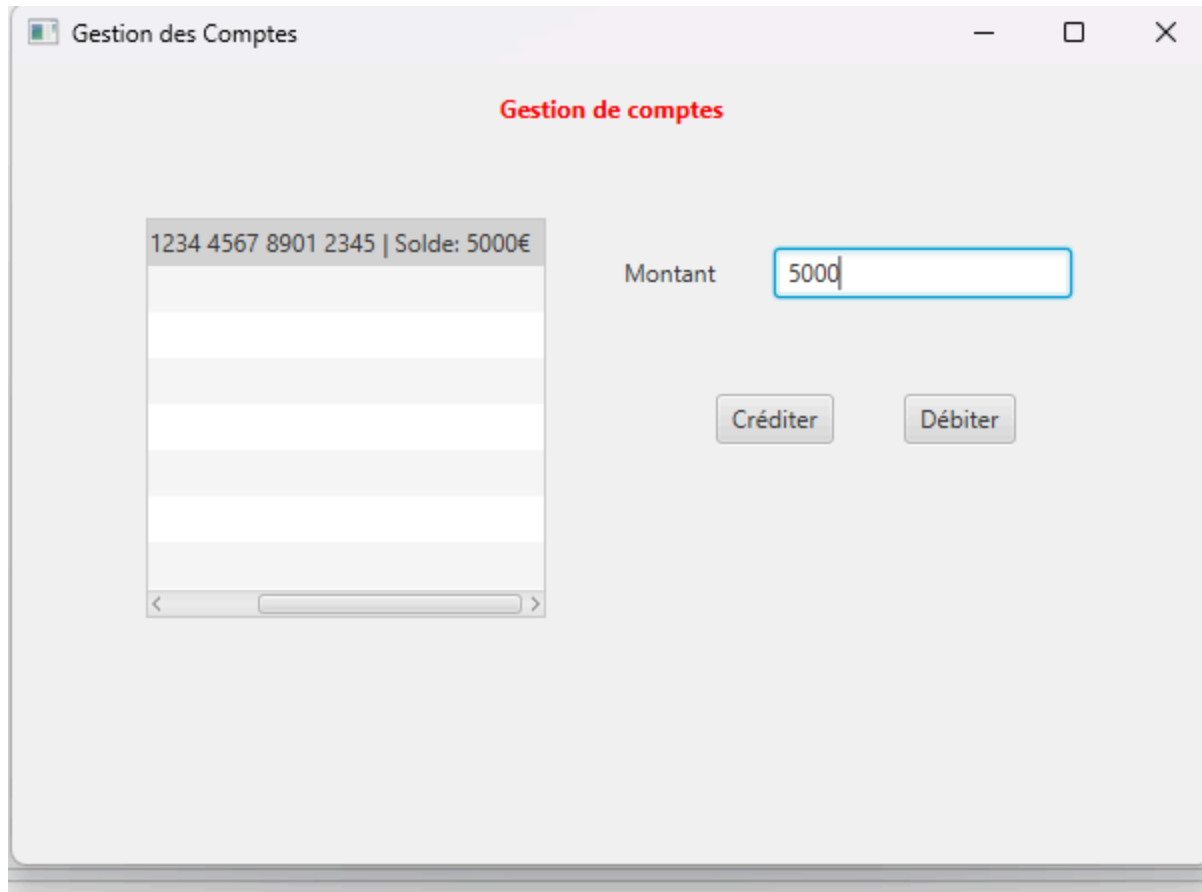
N° CB 1234 4567 8901 2345

Découvert Max 2500 Taux

SOLDE 5000

CREER

Finalement, nous allons tester la gestion de compte avec le nouveau compte que l'on vient de créer, nous revenons sur la page d'index et cliquons sur "Gestion compte", on sélectionne le compte qui a été créé et on décide du montant que l'on veut créditer/débiter, dans notre cas nous allons d'abord créditer 5000 euros sur le solde de Jacques :



Résultat :

1234 4567 8901 2345 | Solde: 10000€

 <- ça a marché

Maintenant nous allons débiter de 7000 euros :

1234 4567 8901 2345 | Solde: 3000€

 <- ça a aussi marché

3) Conclusion

En conclusion, ce TP m'a appris ce qu'était le polymorphisme et la façon dont on l'utilise en Java et sur PowerAMC, ça permet d'en apprendre plus sur la façon dont marchent les liens d'héritages des attributs de plusieurs classes différentes. De plus j'ai aussi pu utiliser ce système pour créer une application de simulation bancaire où l'on peut créer ou gérer des clients et des comptes.

IMPORTANT : Pour ce TD, suite à une erreur de compréhension, j'ai au final décidé de garder le sujet du TD papier sur lequel j'ai travaillé en Java mais j'ai tout de même rajouté l'exercice 13 et 14 du TD7 présent sur le site (le 13 est plus complet sur le site que sur le papier même si en soit je n'ai pas réussi à le faire entièrement durant ce TD).